Who submitted this workload?

Karta exporter PoC | Verified 16 October 2026 | Eight fresh execution cases

What is implemented and proved

A configurable CEL expression reads submitter metadata from the workload and its Pods. The exporter resolves those values and adds `submitted_by` to its existing Pod mapping and workload status metrics. Optional admission captures the authenticated Kubernetes principal for direct submissions. Karta's CRD and SDK are unchanged.

Submission	What ran	Observed submitted_by
Direct Kubernetes	Deployment; finite Job	alice@example.test
Run:AI API	Finite training; HTTP inference fixture	test@run.ai
OSMO API	Single task; leader/worker group	alice, with the email domain lost
NVCF / NVCT APIs	HTTP function; finite task	Empty: no human identity in inspected metadata

All eight cases were freshly submitted against the updated exporter. The direct and training Jobs completed, both OSMO workflows completed, both HTTP fixtures returned 200, and NVCT reached COMPLETED at 100%. These are execution and attribution tests in a fake-GPU lab. The commands do not perform model training, model inference or GPU computation.

What the metrics look like

An observed sample, with structural labels omitted for readability:

```
karta_workload_status{
  workload="cel-label-training", phase="Completed",
  submitted_by="test@run.ai"
} 1
```

The value is a phase indicator, not a user count. The same submitter is present on the workload's `karta_pod_workload_info` samples. Full labels and resource UIDs are retained in the linked evidence.

Contract	Behavior
Attribution disabled	Existing three metric families and labels unchanged
Attribution enabled	Only submitted_by added to karta_pod_workload_info and karta_workload_status
Health	karta_exporter_observation_healthy unchanged
Earlier companion metrics	Removed; no new submitter metric families
Nonempty submitted_by	Raw configured identifier; may be shortened, unverified or a service
Empty submitted_by	Missing, conflicting, invalid or unhealthy attribution; never a guessed user

The test user authenticated with a real short-lived Kubernetes client certificate, without impersonation. Run:AI used its login API. OSMO used development identity headers. NVCF/NVCT used local service credentials. These different authentication scopes must not be presented as equivalent proof of a human identity.

The configuration and who owns it

The cluster administrator selects trusted metadata keys. The platform supplies their values. CEL selects a candidate from each observed object; the exporter combines the selected workload root and its currently attributed Pods. Nothing calls a platform API or identity directory during a scrape.

This is the complex expression deployed in the audit. The first annotation matches the optional admission handler. The simple ternary version is also implemented and tested.

```
exporter:
  enabled: true
  attribution:
    admission:
      enabled: true
    submittedByExpression: |
      (
        [
          "workload.dgxc.nvidia.com/submitter-id",
          "run.ai/submitted-by",
          "workflow.example.com/submitter",
          "platform.example.com/username",
          "platform.example.com/created-by"
        ]
        .filter(key, key in annotations && annotations[key] != "")
        .map(key, annotations[key])
        +
        [
          "identity.example.com/submitted-by",
          "osmo.submitted_by",
          "workflow.example.com/submitter",
          "platform.example.com/username",
          "platform.example.com/created-by"
        ]
        .filter(key, key in labels && labels[key] != "")
        .map(key, labels[key])
        + [""]
      )[0]
```

The lists define precedence: the first nonempty annotation wins before any label. Dots in literal map keys need no backslash. Conflicting keys inside one object can therefore be hidden by this expression. Different nonempty values across the selected root and Pods clear the resolved label.

Admission defaults to namespaces labeled `karta.run.ai/capture-submitter=true` and explicitly selected Deployments, Jobs and Pods. It uses the operator's TLS webhook; the tested cluster does not expose native MutatingAdmissionPolicy. Existing metadata is preserved. Conflicting captured fields are rejected, and captured fields cannot be edited afterward within that scope.

Simple example and complex example. Merge these options into existing installation values. Both legacy configuration forms remain supported; ambiguous old/new combinations are rejected.

Direct kubectl: authenticated identity without overwriting

A one-hour client certificate was issued for `alice@example.test`. A test Role permitted namespaced Deployment and Job submissions. `kubectl auth whoami` confirmed that principal; the submission did not use `--as`.

```
kubectl --kubeconfig "$DEMO_KUBECONFIG" auth whoami
kubectl --kubeconfig "$DEMO_KUBECONFIG" \
  -n karta-label-direct apply -f direct-deployment.json
kubectl --kubeconfig "$DEMO_KUBECONFIG" \
  -n karta-label-direct apply -f direct-job.json
```

Both requests used `busybox:1.37.0`. The Deployment ran a 3,600-second sleep and was removed after capture. The Job slept for 300 seconds, used `restartPolicy: Never`, and completed at 08:26:40 UTC. The requests included a team label and a note to verify preservation.

The resulting root metadata, shortened to the relevant fields:

```
kind: Job
metadata:
  name: cel-label-job
  labels: {team: demo}
  annotations:
    review.example.com/note: keep-me
    workload.dgxc.nvidia.com/submitter-id: alice@example.test
    workload.dgxc.nvidia.com/identity-source: kubernetes:attribution-lab
    workload.dgxc.nvidia.com/submitter-type: unknown
```

```
Deployment cel-label-deployment -> ReplicaSet -> Pod
Job cel-label-job -> Pod
```

Admission records the root's request principal. The controller-created Pods have no copied identity annotation. CEL reads the canonical root annotation; the resolved root value is inherited onto the Pod metric.

```
karta_workload_status{
  workload="cel-label-job", phase="Completed",
  submitted_by="alice@example.test"
} 1
karta_pod_workload_info{
  workload="cel-label-job", pod="cel-label-job-t6bmq",
  submitted_by="alice@example.test"
} 1
```

A create request supplying `bob@example.test` was rejected rather than overwritten. The authenticated user was also blocked from changing the captured root identity. A separate live test added a conflicting Pod annotation: both metrics changed from Alice to empty, then recovered Alice when it was removed. Observation health remained 1 throughout that conflict test.

Gap: admission knows the Kubernetes principal, which can be a service or an impersonated identity in other requests. It does not prove a human, resolve a directory username, backfill existing objects, or recover the user behind a platform service account.

Run:AI: submit through the API, inherit from the workload

Login through `/api/v1/token` was followed by authenticated POSTs to `/api/v1/workloads/trainings` and `/api/v1/workloads/inferences`. The saved request templates retain the actual specification and replace only environment-specific project and cluster IDs.

```
{
  "name": "cel-label-training",
  "projectId": "<project-id>", "clusterId": "<cluster-id>",
  "spec": {
    "image": "busybox:1.37.0", "command": "sh",
    "args": "-c \"echo attribution-start; sleep 300; echo attribution-complete\"",
    "backoffLimit": 0, "restartPolicy": "Never",
    "compute": {"cpuCoreRequest": 0.05, "cpuCoreLimit": 0.2,
      "cpuMemoryRequest": "64Mi", "cpuMemoryLimit": "128Mi",
      "gpuDevicesRequest": 0}
  }
}
```

Inference used the same bounded CPU/memory settings, BusyBox HTTP on port 8080, and one minimum/maximum replica. It returned HTTP 200 through the Knative revision proxy. The test project has zero GPU quota.

```
Training:  Job cel-label-training -> Pod
Inference: Knative Service cel-label-inference
           -> Configuration -> Revision -> Deployment -> ReplicaSet -> Pod
WorkloadRunner objects also existed; they had no submitter metadata.
```

Resulting object	Identity metadata	Outcome
Training Job	run.ai/submitted-by: test@run.ai	Completed at 08:25:44 UTC
Training Pod	user: test@run.ai	Succeeded
Knative Service	run.ai/submitted-by: test@run.ai	Ready
Inference Pod	No submitter annotation	Running; HTTP 200

The deployed expression selects `annotations["run.ai/submitted-by"]` from the root. It deliberately does not select the generic Pod `user` key. Both Pod metrics inherit the root result, including inference whose Pod has no identity metadata.

```
karta_workload_status{
  workload="cel-label-training", phase="Completed",
  submitted_by="test@run.ai"
} 1
karta_pod_workload_info{
  workload="cel-label-inference", workload_kind="Service",
  submitted_by="test@run.ai"
} 1
```

Gaps: this verifies the lab account, not normalization to an agreed NVIDIA username. HTTP serving is not model inference, and external ingress was not tested. Metrics were verified with the namespace-scoped audit exporter and an exact WorkloadRunner read role. Fleet-wide permission coverage and every Run:AI submission mode remain unproven.

OSMO: execution succeeds, but the identity is shortened

POST `/api/pool/default/workflow` received the workflow YAML in its `file` field. Development headers supplied `alice@example.test`. Both requests selected the existing audit platform explicitly; no default platform was changed.

```
workflow:
  name: cel-label-single
  resources:
    default:
      cpu: 1
      memory: 512Mi
      storage: 1Gi
      platform: <audit-platform>
  tasks:
  - name: worker
    image: ubuntu:24.04
    command: [/bin/sh, -c]
    args: ["echo attribution-start; sleep 300; echo attribution-complete"]
```

The group request contained a leader and worker with the same command and resources. The first pair completed; a repeated pair confirmed metrics after observation converged. In the repeated pair all three tasks had exit code 0.

```
OSMO workflow API: cel-label-group-2, submitted_by=alice@example.test
  leader Pod c47243997abb4ec9-fb47d19472b34eb6
  worker Pod c47243997abb4ec9-96a05d548f4f4ff9
Each Pod label: osmo.submitted_by=alice
Karta selection: each task Pod is its own workload root.
```

Repeated workflow	API outcome	Completed UTC
cel-label-single-2	COMPLETED; worker exit 0	08:41:17
cel-label-group-2	COMPLETED; leader and worker exit 0	08:41:20

CEL falls through to `labels["osmo.submitted_by"]`. An observed single-task sample is:

```
karta_pod_workload_info{
  workload="e5d1abe4ce374717-e923b371aa294f01",
  workload_kind="Pod", submitted_by="alice"
} 1
karta_workload_status{
  workload="e5d1abe4ce374717-e923b371aa294f01",
  phase="Running", submitted_by="alice"
} 1
```

These samples were captured while running, before completion and Pod cleanup. Both group Pods also emitted `alice`. An earlier capture had empty labels while observation failed; the repeated capture had healthy observations and populated labels. No persistent OSMO exporter defect was established by that transient result.

Gaps: the API retained the full email, but the Pod label lost its domain. Earlier edge tests showed different email domains collapsing to the same `alice`, and a one-character user producing no label. CEL cannot restore lost information. A platform-controlled full-identity annotation is needed. No Kubernetes workflow root existed in this installation, so these metrics do not yet represent one combined OSMO workflow. Production OIDC was not exercised.

NVCF and NVCT: workload IDs are not a human submitter

The local APIs received a container function and a finite task. NGC was used only for image pulls. No image or result was uploaded.

```
POST /v2/nvcf/functions
  name: cel-label-function
  image: nvcr.io/nvidia/omniverse/simple-nginx:1.0.2
  inference: HTTP port 80, path /
  health: HTTP /, expected status 200
POST /v2/nvcf/deployments/functions/{id}/versions/{version}
  gpu: TESLA, instanceType: NCP.GPU.TESLA_1x
  minInstances: 1, maxInstances: 1

POST /v1/nvct/tasks
  name: cel-label-task
  image: nvcr.io/nvidia/cuda:12.6.3-base-ubuntu22.04
  maxRuntimeDuration: PT5M, resultHandlingStrategy: NONE
  command: write progress, sleep 180, write 100%, exit
```

Execution	Result
Function	ACTIVE; GET invocation returned HTTP 200 and 896 bytes
Task	RUNNING then COMPLETED, 100%, at 08:49:06 UTC

Both new Pods initially lacked CPU capacity. Only their primary container CPU request and limit were resized from 1 CPU to 100m. Keeping both equal preserved Guaranteed QoS; the first request-only resize had been rejected. Shared platform resources and settings were unchanged.

```
Function/task API record -> ICMSRequest -> Pod
The Pod's owner reference has no controller: true.
Karta therefore selects the Pod itself as the workload root.
Metadata exposes workload/account identifiers, not a human submitter.
```

None of the configured submitter keys existed on the inspected Pod or ICMSRequest. CEL returned empty. The observation was healthy, so this case proves missing metadata rather than a stale scrape.

```
karta_pod_workload_info{
  workload="0-sr-df24444a-8807-4fc9-89c6-534c7a701643",
  workload_kind="Pod", submitted_by=""
} 1
karta_workload_status{
  workload="0-sr-a7121b89-f78c-4a0a-84b1-6cecdf0ab202-task",
  phase="Running", submitted_by=""
} 1
```

Gaps: CEL cannot turn an account/function/task ID into the original person. The platform must pass a trusted original principal to Kubernetes. Local service credentials prove the service account path, not the human behind it. The non-controller link also prevents a function-wide root across replicas. Hosted NVCF, Helm-based functions, delegation and actual GPU compute were not tested. A prior malformed-request cancellation issue remains an upstream/platform gap; it did not recur in these successful fixtures.

Bugs, corrections and verification

The first three corrections predate this label-only change. The new eight-case rerun includes all of them. They are exporter/owner-observation fixes, not changes to the Karta CRD.

Finding	Fix and evidence
Valid cluster-scoped owner links rejected	Follow Kubernetes scope rules; keep UID checks and reject invalid cross-namespace links. Earlier same-UID capture recovered 35 workloads.
Cluster roots used an incorrect cache key	Use Kubernetes cache keys for publication, attribution, retries and deletion fences. Regression tests reproduce the old failures.
Full and metadata-only DaemonSet responses differed	Verify through the same API representation as the informer. Earlier same-UID capture recovered 21 stale Pod observations.
Prototype emitted two extra submitter families	Remove them; add only submitted_by to the two existing families. Contract tests verify the exact label delta and unchanged defaults.
Captured input could be overwritten at admission	Reject conflicting supplied values; preserve unrelated metadata. Live spoof and update requests were denied.
Root/Pod disagreement could misattribute usage	Clear the resolved label on conflict. Live Alice -> empty -> Alice recovery passed with health 1.
Expression and legacy settings could be ambiguous	Reject simultaneous forms; render and test both supported forms, scoped TLS webhooks and rollout checksum.

What passed

Exporter `make check` passed, including race tests, identity lifecycle tests and chart tests. Operator `make check` passed, including envtest integration on Kubernetes 1.31.0. Root `make check` passed against the clean implementation commit. Helm lint/render validation and the existing Prometheus rule tests passed. Fifteen explicit chart combinations were checked; fourteen are retained as Go regression cases.

Tests cover missing keys, invalid types, evaluation budgets, metadata changes, deletion/recreation by UID, stale associations, unhealthy output, conflicting roots/Pods and admission preservation. Both supplied CEL examples are evaluated against platform-shaped fixtures. Those unit fixtures supplement the live submissions; they do not replace them.

The isolated operator and both audit exporters were Ready after cleanup. The direct certificate Role/RoleBinding and CSR were removed. Run:AI fixtures, the function, completed task and temporary local API keys were deleted through their APIs. Existing platform installs, OSMO platform defaults and the public demo were preserved. New audit binaries and a separately named CEL ConfigMap remain deployed; previous binaries/configuration were retained.

The label-only implementation is commit `49ff26cc`. The lab ran compiled Linux binaries on a temporary volume, not a published release image. No 100,000-workload identity benchmark or production release acceptance is implied.

Implementation and test sources and sanitized execution evidence include the full requests, metadata, metric labels, timestamps and binary hashes. Earlier recovery snapshots are retained separately with their original build scope.

Read the results without hiding the gaps

In Prometheus or Grafana Explore, show the raw mapping and active phases:

```
karta_pod_workload_info{namespace="runai-attribution-lab"}

karta_workload_status{namespace="runai-attribution-lab"} == 1
```

Count current workloads by the configured identifier, deduplicating phase samples:

```
count by (submitted_by) (
  max by (namespace, workload_uid, submitted_by) (karta_workload_status)
)
```

Keep the empty identifier visible and inspect `karta_exporter_observation_healthy` alongside it. This query counts observed workload roots, including Pod roots. Equal raw usernames from different identity authorities are not proof of the same person. Add the same cluster label to every grouping key when combining clusters.

Remaining gaps across all platforms

Area	Limit or required decision
Identity agreement	Agree the NVIDIA username and authority/tenant mapping. No directory validation, normalization or human/service distinction is exposed by this single label.
Empty versus partial	Empty conflates missing, conflict and evaluator failure. Nonempty can be lossy. The old status/source/type companion labels are intentionally absent.
Trust and precedence	Metadata outside protected scopes can be forged. The first-key CEL expression can hide same-object conflicts. Metrics are not an authorization mechanism.
Observation coverage	Only watched kinds with read RBAC are visible. Resolution combines the selected root and Pods, not intermediate owners or platform databases. Extra watches do not create a workload hierarchy.
Admission coverage	Defaults omit other controllers/CRDs; existing objects are not backfilled. System-created owned children are skipped. Fail-closed admission affects availability in its selected scope.
Freshness and history	Watch/verification delay is possible. Metadata edits change current labels; removed resources disappear. Never join historical GPU samples to today's mapping.
Configuration	Rules compile at startup; Helm changes roll the exporter. Editing a mounted ConfigMap does not hot-reload it. Multiple served versions of one kind remain unvalidated.
Scale and release	No 100,000-workload attribution load test, fleet-wide RBAC proof, production authentication matrix or immutable release-image acceptance was performed.

A new label does not add another series per current sample, but identity changes create historical series churn. CEL has bounded configuration, output and evaluation cost; those limits are not a fleet-scale benchmark.